

Table of contents

Modélisation et Vérification de Logiciels	2
Introduction	2
Partie 1 : Concepts de base de la modélisation et vérification	2
Programme et spécifications	2
Définition d'un programme	2
Propriétés fonctionnelles et non fonctionnelles	3
Propriétés non fonctionnelles	3
Propriétés fonctionnelles	3
Spécifications formelles	3
Le problème de la vérification	3
Méthodes de vérification	4
Raisonnement déductif (Deductive Reasoning)	4
Interprétation abstraite (Abstract Interpretation)	4
Model Checking	5
Preuve assistée (Assisted Proof)	5
Partie 2 : Raisonnement déductif et logique de Hoare	6
Introduction à la logique de Hoare	6
Sémantique opérationnelle	6
Définitions formelles	7
Triplets de Hoare et correction partielle	7
Définition des triplets	7
Validité d'un triplet	7
Correction partielle	8
Calcul de la plus faible précondition (WP)	8
Définition récursive de WP	8
Théorème fondamental pour les programmes sans boucle	8
Traitement des boucles : invariants	8
Définition d'un invariant	9
WP avec invariant pour une boucle	9
Traitement des appels de fonction	10
Contrat de fonction	10
WP pour un appel de fonction	10
Correction totale (variant)	11
Définition du variant	11
Théorème de correspondance entre sémantique opérationnelle et axiomatique	12
Énoncé du théorème	12
Idée de la preuve	12

Modélisation et Vérification de Logiciels

Introduction

La question centrale de ce domaine est : **mon système informatique, programme ou logiciel est-il correct ?**

Cette préoccupation est cruciale à toutes les échelles :

- **Systèmes larges** : Internet, systèmes distribués
- **Systèmes embarqués** : IoT, systèmes critiques, automatisation

La vérification porte sur plusieurs aspects fondamentaux :

- **Algorithmes** → leur implémentation sous forme de programmes
 - **Types de données** → représentation et manipulation correctes
 - **Protocoles** → communication et coordination entre composants
-

Partie 1 : Concepts de base de la modélisation et vérification

Programme et spécifications

Définition d'un programme

Un programme est initialement une **séquence de caractères** qui doit respecter les règles syntaxiques vérifiées par un compilateur.

Attention : Un compilateur ne vérifie que la **syntaxe**, pas la correction sémantique !

De cette séquence de caractères, nous construisons un **modèle du programme** – une abstraction mathématique qui représente la sémantique (la signification) du programme.

Sémantique : Signification des constructions syntaxiques. Exemple : **x++** a pour sémantique “x est incrémenté de 1”.

Les modèles formels sont des **objets mathématiques** qui permettent de raisonner sur le comportement des programmes de manière systématique et, dans une certaine mesure, automatisée. Ceci est particulièrement utile pour les systèmes complexes où l'intuition peut faire défaut.

Propriétés fonctionnelles et non fonctionnelles

Propriétés non fonctionnelles

Ces propriétés concernent la **qualité de service** plutôt que le calcul spécifique :

- Temps d'exécution, performance
- Ergonomie, utilisabilité
- Propriétés de sécurité
- Consommation de ressources (mémoire, énergie)

Propriétés fonctionnelles

Ces propriétés spécifient **ce que le programme est censé calculer**.

Exemple : `qsort(T)` doit retourner un tableau `T'` qui est une permutation triée de `T`.

Spécifications formelles

Les spécifications peuvent être exprimées de diverses manières :

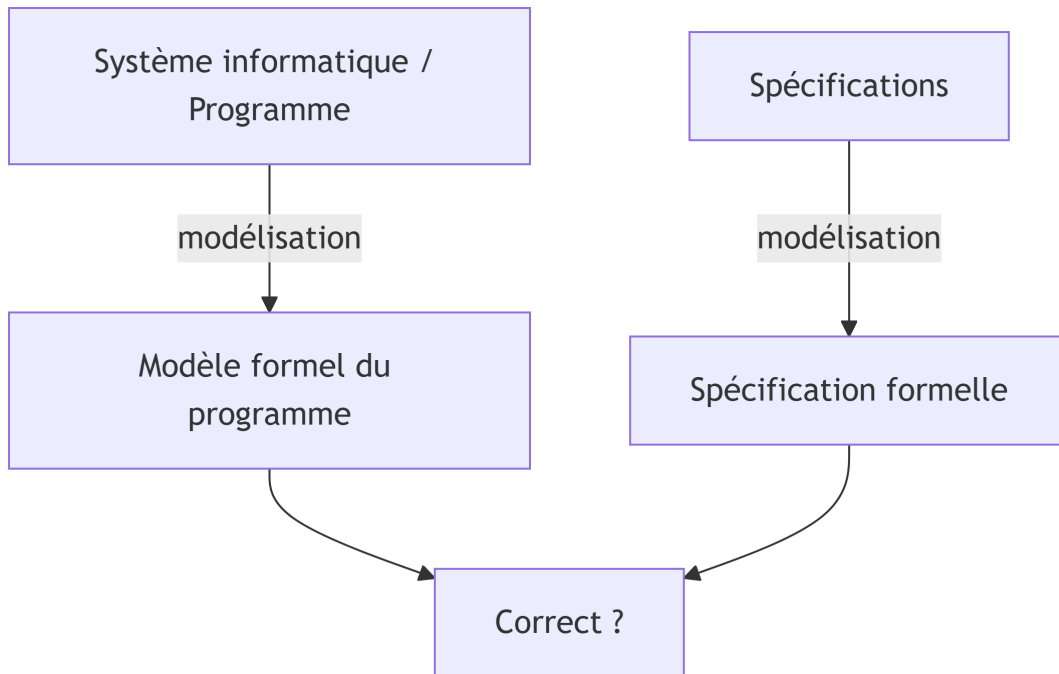
1. **Langage naturel** : Descriptions écrites, souvent ambiguës
2. **Annotations dans le programme** : Commentaires dans le code
3. **Assertions** : Constructions de programmation défensive qui détectent les erreurs à l'exécution
4. **Tests unitaires** : Spécifications par exemples
5. **Spécifications formelles** : Prédicats logiques avec une signification mathématique précise

Exemple de spécification formelle pour un tableau trié :

$$\forall i \in \{0, \dots, n-2\}, t[i] \leq t[i+1]$$

Le problème de la vérification

Nous voulons déterminer si un programme satisfait ses spécifications. Ceci peut être visualisé comme :



Définition : Un programme **satisfait** (ou vérifie) une spécification, noté $\text{programme} \models \text{spécification}$, si pour toutes les exécutions possibles, le comportement du programme est conforme à la spécification.

Méthodes de vérification

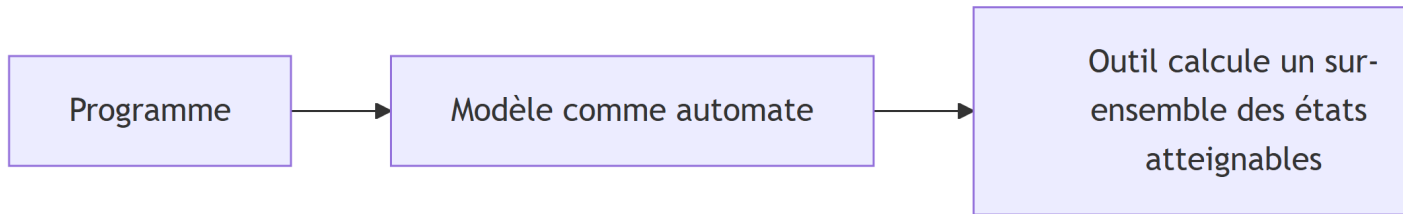
Plusieurs approches existent pour répondre à la question de la correction :

Raisonnement déductif (Deductive Reasoning)

Utilise la logique mathématique pour prouver qu'un programme satisfait sa spécification. La **logique de Hoare** en est un exemple fondamental.

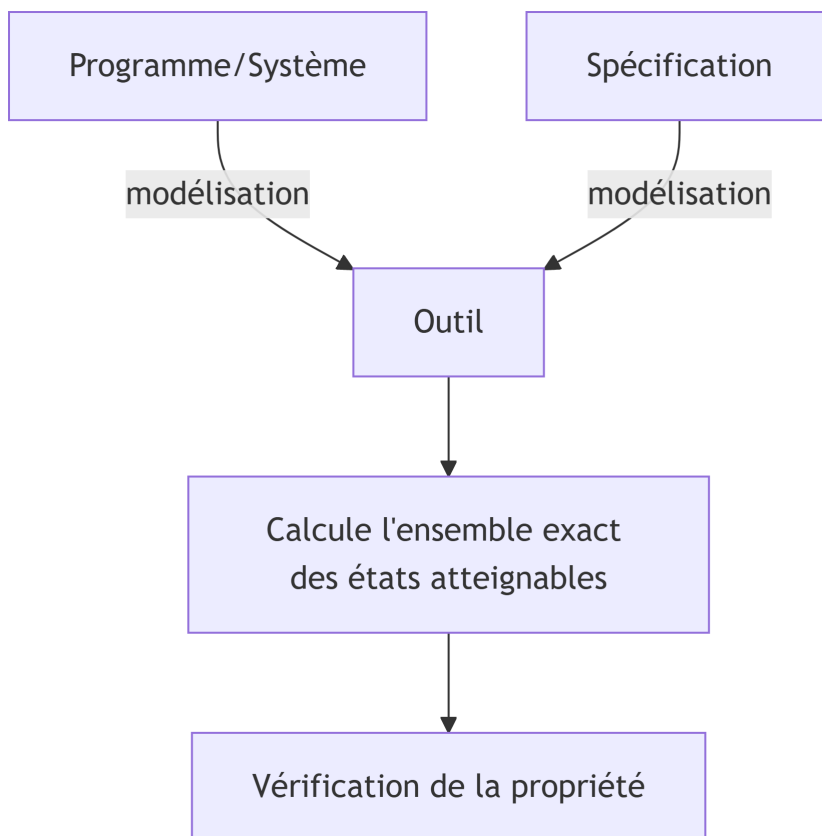
Interprétation abstraite (Abstract Interpretation)

Technique qui calcule une **sur-approximation** des comportements possibles du programme. Si la sur-approximation satisfait la propriété, alors le programme aussi.



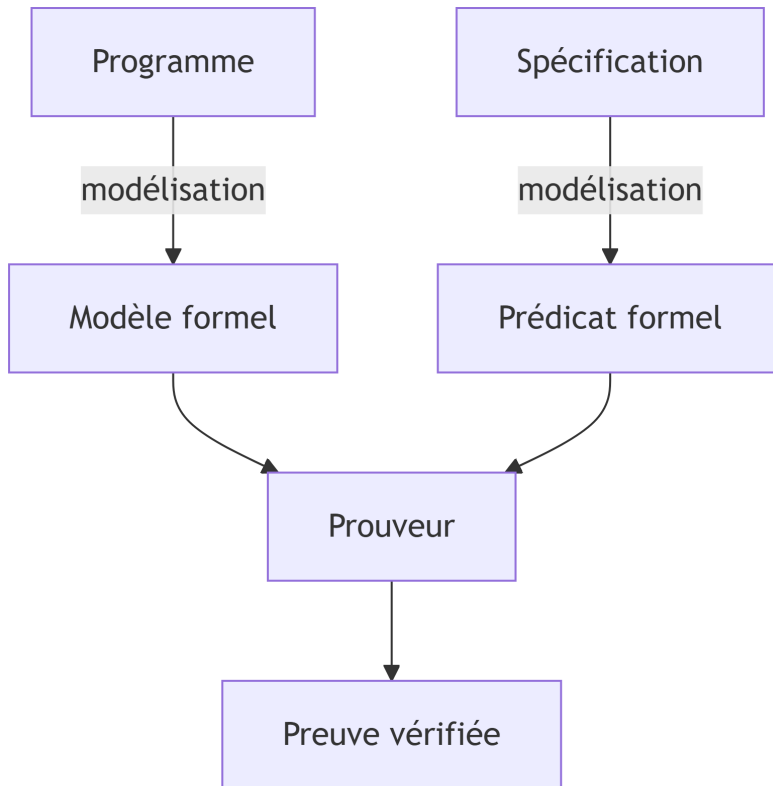
Model Checking

Vérifie automatiquement qu'un modèle fini d'un système satisfait une propriété formelle. Calcule l'**ensemble exact** des états atteignables (dans les limites de calculabilité).



Preuve assistée (Assisted Proof)

Combine l'assistance humaine avec la vérification automatique. L'utilisateur guide la preuve, et la machine vérifie chaque étape.



Partie 2 : Raisonnement déductif et logique de Hoare

Introduction à la logique de Hoare

La **logique de Hoare** (due à C.A.R. Hoare, 1969) est un système formel pour raisonner sur la correction des programmes. Elle permet de dériver mathématiquement qu'un programme satisfait sa spécification.

Idée centrale : Associer à chaque instruction du programme comment elle transforme les propriétés des variables.

Sémantique opérationnelle

Avant d'introduire la logique de Hoare, nous devons définir formellement la signification des programmes. La **sémantique opérationnelle** décrit comment l'état de la mémoire évolue

lors de l'exécution.

Définitions formelles

État mémoire : Noté μ , c'est une fonction qui associe à chaque variable sa valeur.

$$\mu : Var \longrightarrow \mathbb{Z} \cup \{tt, ff\}$$

où Var est l'ensemble des variables du programme, $\mathbb{Z}$ les entiers, et $\{tt, ff\}$ les booléens.

Mise à jour de l'état : $\mu[x \leftarrow c]$ dénote l'état obtenu en modifiant μ pour que x vaille c , les autres variables restant inchangées.

Évaluation d'expression : $Val(e, \mu)$ retourne la valeur de l'expression e dans l'état μ .

Exécution d'instruction : $Mem(c, \mu)$ retourne le nouvel état mémoire après exécution de l'instruction c depuis l'état μ .

Exemple : $Mem(y := 0, \mu) = \mu[y \leftarrow 0]$

Triplets de Hoare et correction partielle

Définition des triplets

Un **triplet de Hoare** a la forme $\{P\} c \{Q\}$ où :

- P est une **précondition** (prédicat logique sur les variables avant exécution)
- c est une instruction ou séquence d'instructions
- Q est une **postcondition** (prédicat logique sur les variables après exécution)

Signification : Si l'état mémoire satisfait P avant d'exécuter c , et si l'exécution de c termine, alors l'état mémoire résultant satisfera Q .

Validité d'un triplet

Un triplet $\{P\} c \{Q\}$ est **valide** si et seulement si :

$$\forall \mu, \mu \models P \implies Mem(c, \mu) \text{ est défini et } Mem(c, \mu) \models Q$$

où $\mu \models P$ signifie “ μ satisfait le prédicat P ”.

Correction partielle

La définition ci-dessus suppose que $Mem(c, \mu)$ est défini, c'est-à-dire que l'exécution de c **termine**. Un triplet valide garantit donc que **si** le programme termine, alors la postcondition est satisfaite. C'est ce qu'on appelle la **correction partielle**.

Remarque importante : Le triplet $\{false\} c \{Q\}$ est toujours valide (car la prémisse est toujours fausse). On dit qu'il est **vacuously true**.

Calcul de la plus faible précondition (WP)

Le principe du **Weakest Precondition (WP)** est de calculer, pour une instruction c et une postcondition Q , la condition la plus faible sur l'état initial qui garantit qu'après exécution de c , Q sera vraie.

Définition récursive de WP

Pour les instructions de base :

1. **Affectation :** $WP(x := t, Q) = Q[x \leftarrow t]$ où $Q[x \leftarrow t]$ est Q dans laquelle chaque occurrence libre de x est remplacée par t .
2. **Séquence :** $WP(c_1; c_2, Q) = WP(c_1, WP(c_2, Q))$
3. **Conditionnelle :**

$$WP(\text{if } b \text{ then } c_1 \text{ else } c_2, Q) = (b \Rightarrow WP(c_1, Q)) \wedge (\neg b \Rightarrow WP(c_2, Q))$$

Théorème fondamental pour les programmes sans boucle

Pour un programme c sans boucle ni appel de fonction, et pour des formules P et Q :

$$\{P\} c \{Q\} \text{ est valide} \iff P \Rightarrow WP(c, Q) \text{ est valide}$$

Autrement dit, P doit impliquer la plus faible précondition de c pour Q .

Traitement des boucles : invariants

Pour les boucles, le calcul de WP n'est pas directement possible (cela résoudrait le problème de l'arrêt, qui est indécidable). L'utilisateur doit fournir un **invariant de boucle**.

Définition d'un invariant

Soit la boucle : `while b do c done`

Un prédicat I est un **invariant** de cette boucle si et seulement si :

$$\{I \wedge b\} c \{I\} \text{ est un triplet valide}$$

Intuition : Si I est vrai avant d'entrer dans le corps de la boucle, et que la condition de boucle b est vraie, alors après exécution du corps, I reste vrai.

WP avec invariant pour une boucle

Avec un invariant I fourni pour la boucle `while b do c done`, on définit :

$$WP(\text{while } b \text{ do } c \text{ done}, Q) = I$$

à condition que :

1. I soit un invariant : $\{I \wedge b\} c \{I\}$ valide
2. En sortie de boucle, I et la négation de la condition impliquent Q : $I \wedge \neg b \Rightarrow Q$

Exemple pratique : Fonction `add(a, b)` avec précondition $b \geq 0$ et postcondition $res = a + b$

```
add(a, b) =  
  res := a;  
  aux := b;  
  while aux > 0 do  
    res := res + 1;  
    aux := aux - 1;  
  done;  
  res
```

Invariant proposé : $I = (res + aux = a + b) \wedge (aux \geq 0)$

Nous devons vérifier :

1. $\{I \wedge aux > 0\} res := res + 1; aux := aux - 1 \{I\}$
2. $I \wedge \neg(aux > 0) \Rightarrow res = a + b$

Le calcul de WP pour le corps de la boucle donne :

$$WP(res := res + 1; aux := aux - 1, I) = I[aux \leftarrow aux - 1][res \leftarrow res + 1]$$

Ce qui se simplifie en $(res + 1) + (aux - 1) = a + b \wedge aux - 1 \geq 0$, équivalent à $res + aux = a + b \wedge aux \geq 1$.

Or $I \wedge aux > 0$ implique $res + aux = a + b \wedge aux \geq 0 \wedge aux > 0$, donc $aux \geq 1$, ce qui implique bien $WP(\dots)$.

Pour la condition de sortie : $I \wedge \neg(aux > 0)$ implique $res + aux = a + b \wedge aux \geq 0 \wedge aux \leq 0$, donc $aux = 0$, d'où $res = a + b$.

Ainsi, I est bien un invariant valide.

Traitement des appels de fonction

Contrat de fonction

Une fonction f peut être associée à un **contrat** $\{\phi\} f \{\psi\}$ où :

- ϕ sont les préconditions sur les paramètres
- ψ sont les postconditions reliant les paramètres et la valeur de retour

WP pour un appel de fonction

Pour un appel $a := f(b)$ dans un contexte où f a pour contrat $\{\phi\} f \{\psi\}$, on a :

$$WP(a := f(b), Q) = \phi[x \leftarrow b] \wedge \forall v, (\psi[x \leftarrow b, res \leftarrow v] \Rightarrow Q[a \leftarrow v])$$

où x est le paramètre formel de f , res sa valeur de retour, et v une variable fraîche.

En pratique, on utilise souvent une version simplifiée :

$$WP(a := f(b), Q) = \phi[x \leftarrow b] \wedge (\psi[x \leftarrow b, res \leftarrow f(b)] \Rightarrow Q[a \leftarrow f(b)])$$

Exemple : Fonction récursive `sum(n)` qui calcule $\sum_{i=1}^n i$

```

sum(n) =
  requires { n >= 0 }
  ensures { res =  $\sum_{i=1}^n i$  }
  if n = 0 then
    res := 0
  else
    res := n + sum(n - 1)
  endif;
res

```

Pour vérifier le contrat, nous devons calculer WP du corps avec la postcondition.

Soit $\phi(n) = n \geq 0$ et $\psi(n, res) = res = \sum_{i=1}^n i$.

Pour le cas $n = 0$: $WP(res := 0, \psi) = (0 = \sum_{i=1}^0 i)$ qui est vrai car $\sum_{i=1}^0 i = 0$.

Pour le cas $n \neq 0$: $WP(res := n + sum(n - 1), \psi)$

Cela devient : $\phi(n - 1)$ (précondition de l'appel) et $\psi(n - 1, sum(n - 1)) \Rightarrow \psi(n, n + sum(n - 1))$.

Ce qui donne : $(n - 1 \geq 0) \wedge (sum(n - 1) = \sum_{i=1}^{n-1} i) \Rightarrow (n + sum(n - 1) = \sum_{i=1}^n i)$.

Puisque $\sum_{i=1}^n i = n + \sum_{i=1}^{n-1} i$, l'implication est vraie.

La vérification complète nécessite aussi de montrer que $n \geq 0 \Rightarrow n - 1 \geq 0$ lorsque $n \neq 0$, ce qui est le cas.

Correction totale (variant)

La correction partielle ne garantit pas la **terminaison**. Pour obtenir la **correction totale**, nous devons ajouter un **variant**.

Définition du variant

Un **variant** est une expression entière V sur les variables du programme telle que :

1. $V \geq 0$ avant chaque itération de boucle ou chaque appel récursif
2. V décroît strictement à chaque itération/appel

Pour les boucles : Pour **while** b **do** c **done** avec invariant I , on exige l'existence d'un variant V tel que :

- $I \wedge b \Rightarrow V \geq 0$
- $\{I \wedge b \wedge V = v_0\} c \{V < v_0\}$ (où v_0 est une variable logique)

Pour les appels récursifs : Pour une fonction $f(x)$ avec précondition $\phi(x)$, on exige un variant $V(x)$ tel que :

- $\phi(x) \Rightarrow V(x) \geq 0$
- Pour tout appel $f(y)$ dans le corps de f , $\phi(x) \Rightarrow V(y) < V(x)$

Exemple : Pour la fonction `sum(n)`, le variant peut être n lui-même.

Nous avons : $n \geq 0 \Rightarrow n \geq 0$ (évident) Et pour l'appel récursif `sum(n-1)` : $n \geq 0 \wedge n \neq 0 \Rightarrow n - 1 < n$ (vrai)

Ainsi, la terminaison est garantie.

Théorème de correspondance entre sémantique opérationnelle et axiomatique

Ce théorème fondamental établit l'équivalence entre la définition opérationnelle de la validité d'un triplet de Hoare et sa caractérisation par WP .

Énoncé du théorème

Pour tout programme c (avec ou sans boucles, avec traitement par invariants et variants) et pour toutes formules P et Q :

$$\{P\} c \{Q\} \text{ est valide (selon la sémantique opérationnelle)} \iff P \Rightarrow WP(c, Q) \text{ est valide}$$

où WP est étendu aux boucles avec invariants comme décrit précédemment.

Idée de la preuve

La preuve procède par **induction structurelle** sur le programme c . Nous présentons les cas clés :

Cas de l'affectation

Pour $c = x := t$, nous devons montrer :

$$\{P\} x := t \{Q\} \text{ valide} \iff P \Rightarrow Q[x \leftarrow t] \text{ valide}$$

Preuve : Par définition opérationnelle, $\{P\} x := t \{Q\}$ est valide ssi pour tout μ , si $\mu \models P$, alors $Mem(x := t, \mu) = \mu[x \leftarrow val(t, \mu)] \models Q$.

Or, par une propriété sémantique fondamentale : $\mu \models Q[x \leftarrow t]$ ssi $\mu[x \leftarrow val(t, \mu)] \models Q$.

Donc la condition équivaut à : si $\mu \models P$, alors $\mu \models Q[x \leftarrow t]$, c'est-à-dire $P \Rightarrow Q[x \leftarrow t]$.

Cas de la séquence

Pour $c = c_1; c_2$, en supposant l'hypothèse d'induction pour c_1 et c_2 :

$\{P\} c_1; c_2 \{Q\}$ valide
ssi il existe R tel que $\{P\} c_1 \{R\}$ et $\{R\} c_2 \{Q\}$ valides
ssi (par IH) $P \Rightarrow WP(c_1, R)$ et $R \Rightarrow WP(c_2, Q)$
ssi $P \Rightarrow WP(c_1, R)$ et $R \Rightarrow WP(c_2, Q)$ pour un certain R

Or $WP(c_1; c_2, Q) = WP(c_1, WP(c_2, Q))$ est la plus faible telle R .

Donc cela équivaut à $P \Rightarrow WP(c_1, WP(c_2, Q)) = WP(c_1; c_2, Q)$.

Cas de la boucle

Pour $c = \text{while } b \text{ do } c'$ done avec invariant I :

La validité opérationnelle de $\{P\} c \{Q\}$ requiert que pour tout μ avec $\mu \models P$, l'exécution de la boucle termine dans un état μ' tel que $\mu' \models Q$.

Par induction sur le nombre d'itérations et en utilisant le fait que I est préservé par chaque itération, on montre que cela équivaut à :

1. $P \Rightarrow I$ (l'invariant est vrai initialement)
2. $I \wedge b \Rightarrow WP(c', I)$ (l'invariant est préservé)
3. $I \wedge \neg b \Rightarrow Q$ (la postcondition est vérifiée en sortie)

Ce qui est exactement la définition de $P \Rightarrow WP(c, Q)$ pour une boucle, où $WP(c, Q) = I$ sous les conditions 2 et 3.

Ainsi, les deux définitions coïncident, établissant la correspondance entre sémantique opérationnelle et axiomatique à travers le calcul de WP .

Ce cours a présenté les fondements de la modélisation et vérification de logiciels, en se concentrant particulièrement sur le raisonnement déductif et la logique de Hoare. Ces techniques formelles permettent d'établir rigoureusement la correction des programmes, complétant ainsi les méthodes empiriques comme les tests.